

Baseline Architectural Characterization

ELEC 5803

Jesse Levine (101185127)

1 Architectural Characterization of the Baseline Core

Before implementing the hardware specialization, the baseline RV32IM core was characterized using a self-attention kernel. This characterization serves as the empirical justification for moving from a software-only approach to a heterogeneous hardware-software architecture.

1.1 Baseline Kernel Implementation

The benchmarked kernel implements a full self-attention mechanism defined by the following expression:

$$O = \text{softmax}(QK^T)V \quad (1)$$

where Q, K and V represent the Query, Key and Value matrices respectively.

While the standard Transformer attention mechanism utilizes a $\frac{1}{\sqrt{d_k}}$ scaling factor and true division for normalization, this baseline implementation omits the scaling factor and utilizes bit-shifting for normalization to accommodate the integer-only constraints of the RV32IM core. This transition from true division to shifting is a primary source of the numerical degradation observed as N scales (Figure 1(c)).

The kernel is structured into three distinct computational phases to mimic the standard Transformer attention head:

1. **Score Computation (QK^T):** Calculates N^2 dot products between Query (Q) and Key (K) vectors of dimensions (D).
2. **Softmax Normalization:** Performs a three-pass operation per query:
 - (a) **Max-Search:** Scans scores to find the maximum value for numerical stabilization.
 - (b) **Exponentiation:** Computes e^{x-max} using a 7th-order Taylor series expansion to approximate transcendental math on the inter-only core.
 - (c) **Sum & Shift:** Accumulates the exponents and applies a bit-shifting normalization (to avoid division) to generate probabilities (P).

-
3. **Context Computation (PV):** Computes the final weighted sum of the Value (V) vectors based on the generated probabilities.

This kernel is thus representative of a "best-effort" software implementation for an embedded core lacking an FPU, relying heavily on manual fixed-point rounding and saturation logic to maintain stability.

1.2 Computational Scaling

The first experiment measured the instruction count and total clock cycles required to execute a full self-attention head—including QK^T , Softmax, and PV kernels—as the sequence length (N) increased. Using a 7th-order Taylor series approximation for exponentiation in a $Q16.16$ fixed-point format, the kernel was executed for values of N up to 16.

As shown in Figure 1(a) and (b), the system exhibits a clear quadratic relationship between sequence length and computational cost. While $N = 2$ requires only 1,576 instructions, the count escalates to 93,834 at $N = 16$. When factoring in the measured CPI of 5 (Fetch, Decode, Execute, Memory, Write-back) for the HLS-based RV32IM core, the $N = 16$ workload represents approximately 469,170 clock cycles. This "Instruction Wall" demonstrates that the scalar overhead—specifically loop management, address calculation, and branch logic—is unsustainable for real-time transformer inference on a general-purpose pipeline.

1.3 Numerical Degradation and Probability Collapse

Beyond latency, the baseline experiments revealed a critical failure in numerical fidelity. To maintain execution speed and avoid the massive cycle penalty of software division, the kernel utilized the "Shift-Normalization" strategy described in Section 1.1. As illustrated in Figure 1(c), the probability sum collapses as N increases. While $N = 2$ yields a sum of 0.908, by $N = 16$, the sum drops to 0.082. This collapse occurs because the large bit-shifts required to prevent overflow for large summations effectively quantize smaller probability values to zero.

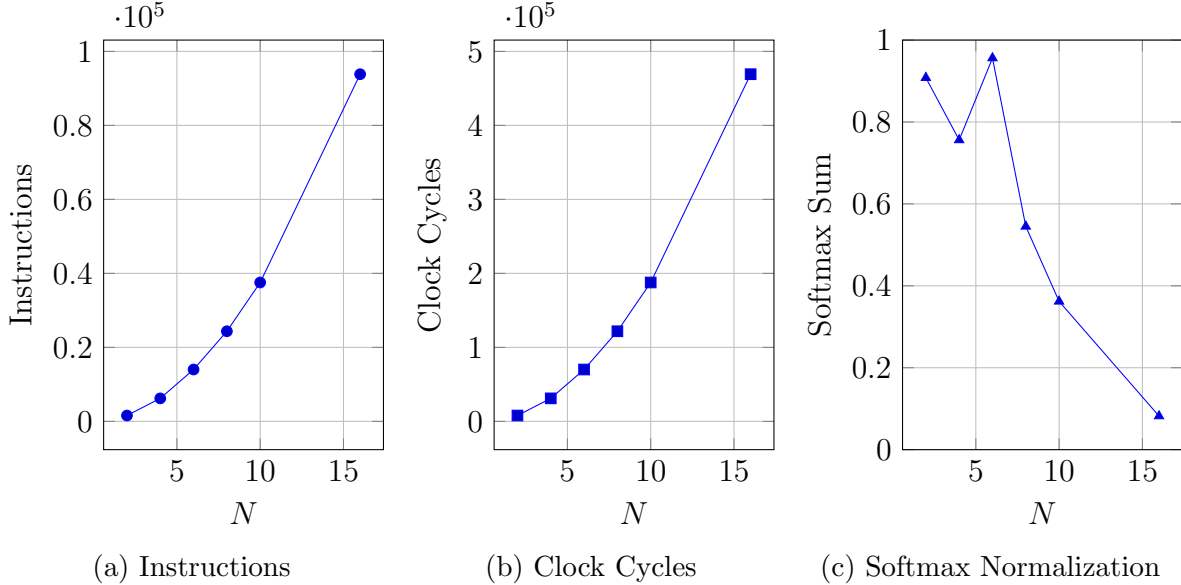


Figure 1: Scaling behaviour of the RV32IM self-attention kernel versus sequence length N .

This numerical instability provides the primary requirement for the hardware accelerator.

2 Proposed Hardware Architecture

The proposed architecture adopts a heterogeneous SoC approach to solve the bottlenecks identified in Section 1. To optimize for end-to-end latency, the design employs a dual-optimization strategy that targets both the throughput of linear matrix operations and the computational complexity of non-linear normalization.

2.1 Hardware-Software Partitioning Strategy

While [1] identifies Softmax as an emerging bottleneck due to its non-linear nature and multiple memory passes, the baseline characterization proved that for a resource-constrained RV32IM core, the "Scalar Tax" (CPI) is an equally critical barrier to performance.

The RISC-V core handles the linear math- QK^T and PV matrix multiplications. Although these are $O(N^2)$ operations, they are mathematically simple (Dot Products) and benefit the most from traditional CPU pipeline optimizations.

Whereas the softmax accelerator hardware block is designed specifically to handle the transcendental e^x logic and the reduction-heavy normalization that causes numerical collapse in software-only implementations.

2.2 Dual-Optimization Objectives

The architecture is designed to balance the trade-offs between instantaneous throughput and total system latency.

By applying HLS pipelining pragmas to the RV32IM core’s main execution loop, I target a reduction in CPI from 5 toward a CPI of 1 (ideal). However, pipelining alone is insufficient if the memory interface remains a bottleneck. In the baseline characterization, the use of a single-port BRAM created a structural hazard: the core could not fetch a new instruction while simultaneously performing a data Load/Store.

To resolve this, the proposed architecture transitions to a Harvard-style memory interface using dual-port BRAM or split Instruction/Data memories. This allows the Fetch and Memory/Execute stages of the pipeline to operate in parallel, ensuring the CPU does not stall during the linear phases (QK^T) of the Attention head.

While the core handles the linear math ($O(N^2D)$) of the workload, the Softmax block utilizes a streaming pipeline to offload the non-linear ($O(N^2)$). By replacing thousands of scalar instructions—which involve high-latency Taylor series math and multiple memory passes—with a hard-wired datapath, the absolute time required to finish the Attention kernel is minimized.

2.3 Softmax Microarchitecture

The proposed accelerator utilizes a hybrid PLA-LUT architecture, transitioning the Softmax operations from a sequence of high-latency scalar instructions into a high-throughout hardware pipeline.

Unlike the 7th-order Taylor series used in the baseline software kernel—which requires

multiple serial multiplications—the PLA-LUT approach approximates the exponential function e^x using a series of linear segments defined by the equation $y = mx + b$.

The input range is partitioned into segments, with the optimal slope (m) and intercept (b) for each region stored in a small, high-speed BRAM or Distributed RAM lookup table. For each element, the hardware fetches the corresponding coefficients and performs a single Multiply-Accumulate (MAC) operation using the FPGA’s dedicated DSP slices. To restore numerical stability, the Sum & Shift logic is replaced with a Reciprocal LUT and a fixed-point multiplier. This allows the accelerator to perform precise division-like normalization in a single clock cycle, preventing the probability sum collapse seen in Figure 1(c).

Furthermore, the choice of PLA-LUT microarchitecture provides a scalable foundation that can be further optimized to meet more aggressive latency targets.

Because the PLA-LUT units are area-efficient, the design can be vectorized by instantiating multiple parallel hardware lanes. This allows the system to process multiple elements of the $N \times N$ attention matrix simultaneously, effectively dividing the $O(N^2)$ latency by the number of hardware lanes.

Furthermore, the datapath is designed to be fully pipelined. While the normalization stage is finalizing the result for element i , the PLA unit is already calculating the exponential for element $i+1$. This ensures a consistent throughput of one result per clock cycle, regardless of the mathematical complexity.

Finally, by utilizing an AXI-Stream or direct-access BRAM interface, the microarchitecture bypasses the CPU’s instruction-fetch bottleneck entirely. This decoupling allows the accelerator to run at its maximum hardware frequency, independent of the RISC-V core’s CPI limitations.

3 AI Usage

ChatGPT and Gemini were both used as an iterative design assistant to support debugging, architectural reasoning, and workload selection during development of the RISC-V attention kernel and accelerator exploration. I actively questioned and clarified issues such as instruction versus cycle measurements, CPI interpretation, memory layout correctness, and the numerical behavior of fixed-point Softmax, using the dialogue to refine my experimental methodology. Through this process, I transitioned from earlier MLP-based experiments to scalable attention kernels that more accurately expose quadratic complexity and architectural bottlenecks. All implementation choices, measurements, and conclusions were independently validated, with AI serving primarily as a tool for conceptual clarification and faster iteration rather than a source of final results.

NOW ITS TIME TO WORK ON THE ACCELERATOR AND CORE OPTIMIZATION

References

- [1] R. Kim, D. Lee, J. Kim, J. Park, and S. E. Lee, “Hardware accelerator for approximation-based softmax and layer normalization in transformers,” *Electronics*, vol. 14, no. 12, 2025. [Online]. Available: <https://www.mdpi.com/2079-9292/14/12/2337>